

目录



- [01、为什么 Agent 需要人工...](#)
- [02、危险操作怎么判断](#)
- [03、审批请求怎么设计](#)
- [04、审批结果怎么设计](#)
- [05、终端交互怎么实现](#)
- [06、拦截层：HitlToolRegistry](#)
- [07、集成到 Main.java](#)
- [08、流式渲染器的冲突处理](#)
- [09、HITL 的几个设计权衡](#)
 - [权衡一：默认关闭](#)
 - [权衡二：拒绝原因回传给 Agent](#)
 - [权衡三：APPROVED_ALL 的...](#)
 - [权衡四：接口隔离测试](#)
- [10、单元测试](#)
- [11、完整跑一遍](#)
- [ending](#)

目录

[01、为什么 Agent 需要人工...](#)

[02、危险操作怎么判断](#)

[03、审批请求怎么设计](#)

[04、审批结果怎么设计](#)

[05、终端交互怎么实现](#)

[06、拦截层：HitlToolRegistry](#)

[07、集成到 Main.java](#)

[08、流式渲染器的冲突处理](#)

[09、HITL 的几个设计权衡](#)

[\[权衡一：默认关闭\]\(#\)](#)

[\[权衡二：拒绝原因回传给 Agent\]\(#\)](#)

[\[权衡三：APPROVED_ALL 的...\]\(#\)](#)

[\[权衡四：接口隔离测试\]\(#\)](#)

[10、单元测试](#)

[11、完整跑一遍](#)

[ending](#)

原创

没有 HITL 人工审批的 Agent，rm -rf 他都敢干。

我们现在的 Agent 是完全信任 AI 的，这等于说把一些危险的动作也交给了 AI。

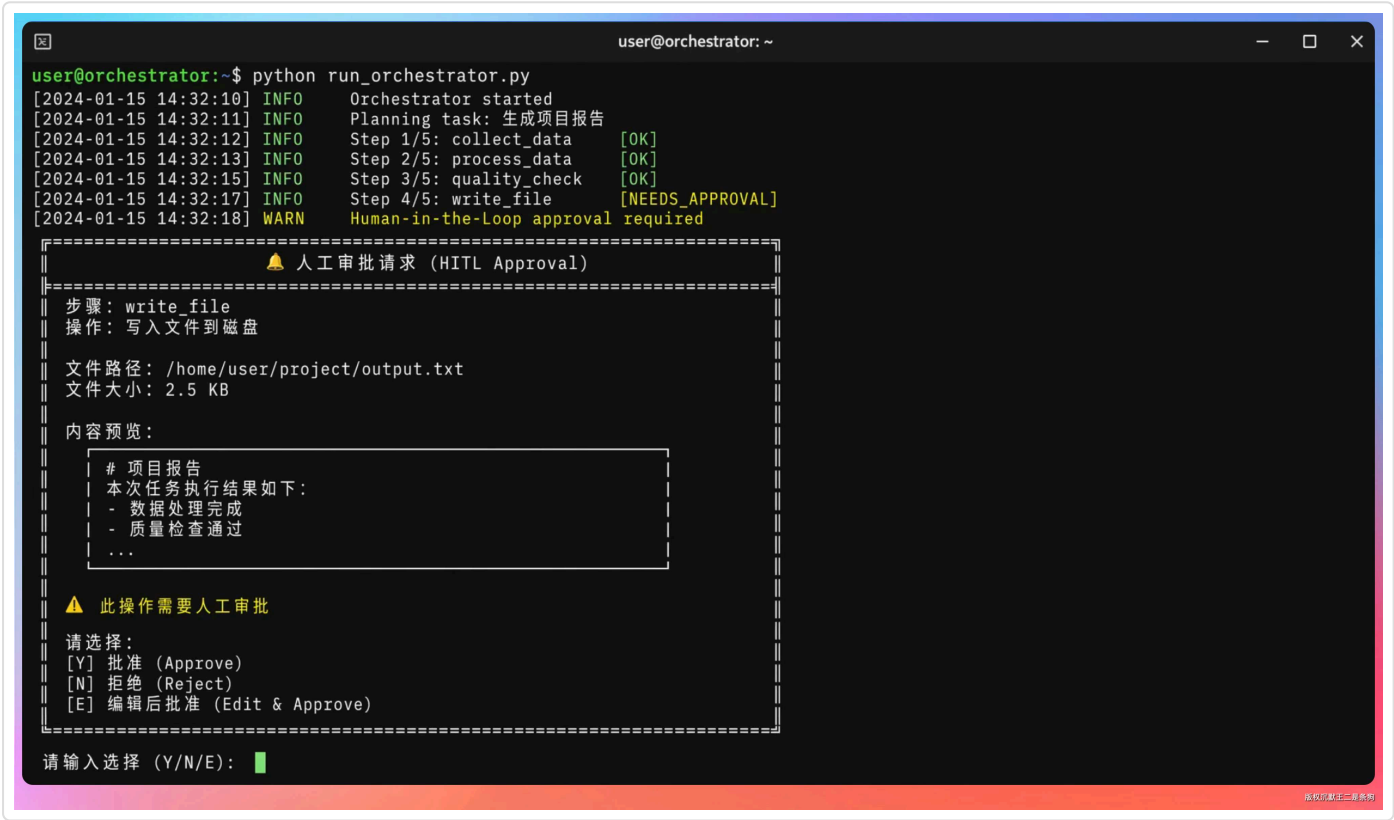
比如说删除一个文件夹。



但真正的 Agent 显然不应该具备这种无限制的权限，在遇到一些危险的操作时候，应该把控制权交给我们人。

于是 HITL，Human-in-the-Loop，人工审批介入出现了。

危险操作执行前，先问一声。



这篇文章会手把手带大家实现完整的 HITL 系统，包括危险策略识别、审批请求格式化、用户交互收集、以及最核心的拦截层设计。

01、为什么 Agent 需要人工审批

先说清楚背景。

PaiCLI 已经有了 ReAct、Plan-and-Execute、Memory、RAG、Multi-Agent，功能挺全的。

但越能干，风险越高。

一个 Agent 在执行任务的时候，可能会：

用 `write_file` 覆写一个你不想动的文件，用 `execute_command` 跑一条不符合预期的命令，用 `create_project` 在磁盘上创建出一堆你不知道的文件夹。

Agent 本身是没有恶意的，但模型有时候会“理解偏差”，把你的意图曲解成另一件事。

目录

[01、为什么 Agent 需要人工...](#)

[02、危险操作怎么判断](#)

[03、审批请求怎么设计](#)

[04、审批结果怎么设计](#)

[05、终端交互怎么实现](#)

[06、拦截层：HitlToolRegistry](#)

[07、集成到 Main.java](#)

[08、流式渲染器的冲突处理](#)

[09、HITL 的几个设计权衡](#)

[权衡一：默认关闭](#)

[权衡二：拒绝原因回传给 Agent](#)

[权衡三：APPROVED_ALL 的...](#)

[权衡四：接口隔离测试](#)

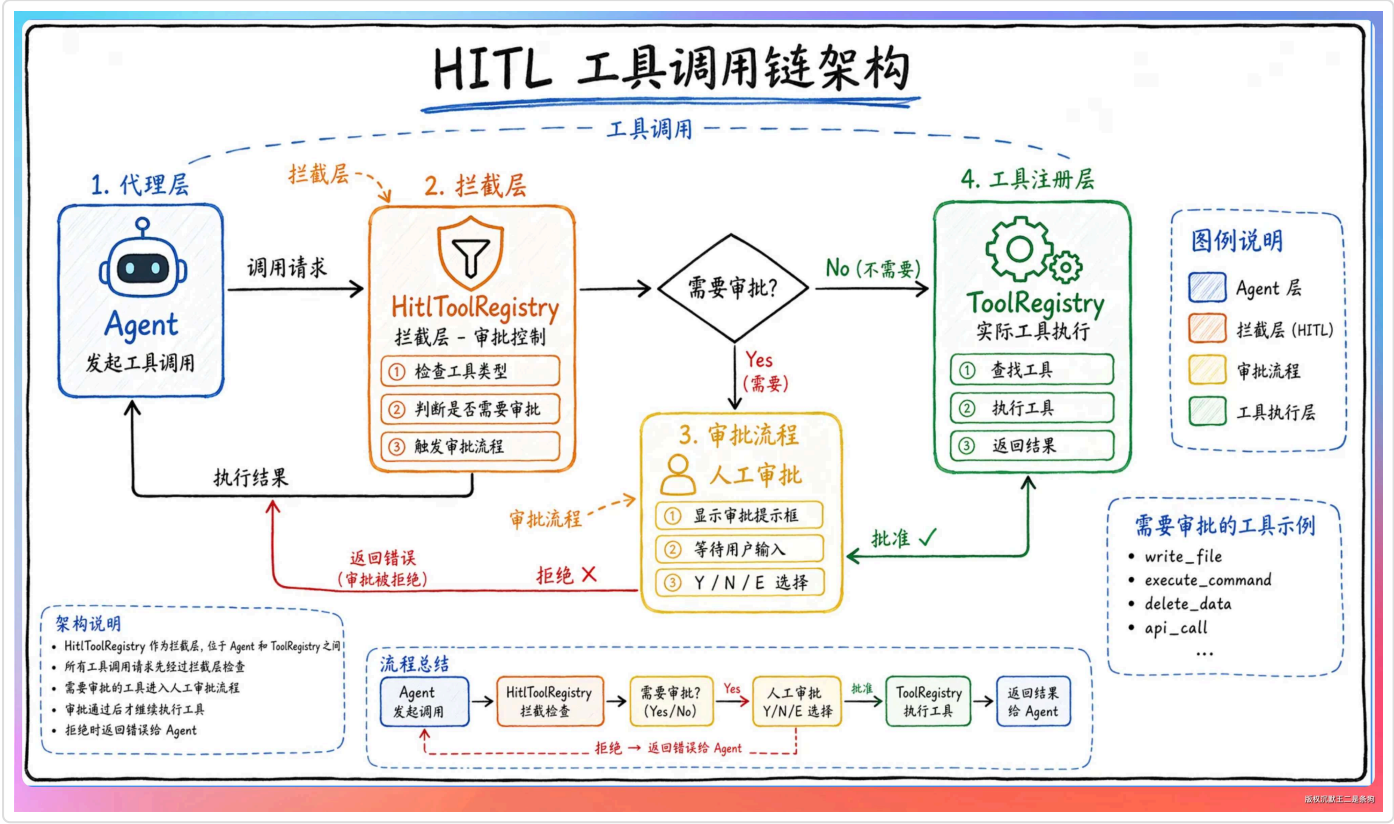
[10、单元测试](#)

[11、完整跑一遍](#)

[ending](#)

特别是在 Plan-and-Execute 和 Multi-Agent 模式下，任务执行的周期一旦很长，执行者就有可能犯下大错。

HITL 的出发点很简单：在执行之前，让人眼瞅一眼，确认没问题再继续。



这不是什么新鲜概念。

Claude Code 里有个 `--dangerously-skip-permissions` 参数，参数名里的 `dangerously` 就说明了一切——默认状态下，执行危险操作前它会停下来等你确认。

Codex 里有个 Approval Mode，也是同样的道理。

工具的设计者意识到，一个有写磁盘和执行命令能力的 Agent，不加任何约束直接跑，风险是很大的。

大模型的幻觉虽然在下降，但永远不会到零。

用户的意图经过提示词翻译成行动，本身就有信息损耗。损耗叠加幻觉，执行就可能跑偏。HITL 不是说 Agent 不可信，而是在模型还没有足够可靠之前，人始终要把控最后一道关。

02、危险操作怎么判断

这里有个很关键的设计决策：危险操作的判断，用静态规则还是动态 LLM 判断？

我选的是静态规则，理由很简单。

动态判断意味着每次调用工具之前，都要问一次 LLM：“这个操作危险吗？”这不仅慢，而且不可靠——今天问出来是危险的，明天可能问出来是安全的。

模型本身有随机性，用它来判断“是否要人工干预”，我们暂时先不考虑。

直接用静态规则写死一个名单。

目录

01、为什么 Agent 需要人工...

02、危险操作怎么判断

03、审批请求怎么设计

04、审批结果怎么设计

05、终端交互怎么实现

06、拦截层：HitlToolRegistry

07、集成到 Main.java

08、流式渲染器的冲突处理

09、HITL 的几个设计权衡

权衡一：默认关闭

权衡二：拒绝原因回传给 Agent

权衡三：APPROVED_ALL 的...

权衡四：接口隔离测试

10、单元测试

11、完整跑一遍

ending

java 复制代码

```
public class ApprovalPolicy {

    // 需要人工确认的工具集合
    private static final Set<String> DANGEROUS_TOOLS = Set.of(
        "write_file",
        "execute_command",
        "create_project"
    );

    public static boolean requiresApproval(String toolName) {
        return DANGEROUS_TOOLS.contains(toolName);
    }

    public static String getDangerLevel(String toolName) {
        return switch (toolName) {
            case "execute_command" -> "🔴 高危";
            case "write_file"      -> "🟡 中危";
            case "create_project"  -> "🟡 中危";
            default                 -> "🟢 安全";
        };
    }

    public static String getRiskDescription(String toolName) {
        return switch (toolName) {
            case "execute_command" -> "将在系统上执行 Shell 命令，可能修改文件、安装软件或影响系统状态";
            case "write_file"      -> "将写入或覆盖文件内容，原有内容将丢失";
            case "create_project"  -> "将在磁盘上创建新目录和文件";
            default                 -> "安全的只读操作";
        };
    }
}
```

read_file、list_dir、search_code 这三个工具是只读操作，不会改变任何东西，不需要审批。

write_file、execute_command、create_project 会写磁盘或跑命令，需要人工确认。

逻辑简单、结果可预期，这才是做 Agent 该有的样子。

目录

[01、为什么 Agent 需要人工...](#)

[02、危险操作怎么判断](#)

[03、审批请求怎么设计](#)

[04、审批结果怎么设计](#)

[05、终端交互怎么实现](#)

[06、拦截层：HitlToolRegistry](#)

[07、集成到 Main.java](#)

[08、流式渲染器的冲突处理](#)

[09、HITL 的几个设计权衡](#)

[权衡一：默认关闭](#)

[权衡二：拒绝原因回传给 Agent](#)

[权衡三：APPROVED_ALL 的...](#)

[权衡四：接口隔离测试](#)

[10、单元测试](#)

[11、完整跑一遍](#)

[ending](#)

高危工具

风险等级：高

审批要求：必须人工审批

execute_command

描述：
执行系统命令

风险：
可能造成系统破坏

审批：
必须人工审批

delete_data

描述：
删除数据

风险：
数据不可恢复

审批：
必须人工审批

system_config

描述：
修改系统配置

风险：
影响系统稳定性

审批：
必须人工审批

中危工具

风险等级：中

审批要求：建议人工审批

write_file

描述：
写入文件

风险：
可能覆盖重要文件

审批：
建议人工审批

api_call

描述：
调用外部API

风险：
数据泄露风险

审批：
建议人工审批

database_update

描述：
更新数据库

风险：
数据一致性风险

审批：
建议人工审批

低危工具

风险等级：低

审批要求：无需审批

read_file

描述：
读取文件

风险：
仅读取,无破坏性

审批：
无需审批

search_data

描述：
搜索数据

风险：
仅查询,无修改

审批：
无需审批

generate_report

描述：
生成报告

风险：
无副作用

审批：
无需审批

风险评估矩阵

工具名称	危险等级	需要审批	自动执行
execute_command	高危	是	否
write_file	中危	是	否
read_file	低危	否	是

配置示例 (JavaScript)

```
1 const RISK_LEVELS = {
2   HIGH: ['execute_command',
3         'delete_data', 'system_config'],
4   MEDIUM: ['write_file', 'api_call',
5           'database_update'],
6   LOW: ['read_file', 'search_data',
7        'generate_report']
8 };
```

关键点

- 静态规则配置
- 基于工具名称判断
- 可自定义危险等级
- 支持动态调整

版权沉默王二是条狗

目录

- 01、为什么 Agent 需要人工...
- 02、危险操作怎么判断
- 03、审批请求怎么设计
- 04、审批结果怎么设计
- 05、终端交互怎么实现
- 06、拦截层：HitlToolRegistry
- 07、集成到 Main.java
- 08、流式渲染器的冲突处理
- 09、HITL 的几个设计权衡
 - 权衡一：默认关闭
 - 权衡二：拒绝原因回传给 Agent
 - 权衡三：APPROVED_ALL 的...
 - 权衡四：接口隔离测试
- 10、单元测试
- 11、完整跑一遍 ending

execute_command 是高危（会跑 shell 命令），write_file 和 create_project 是中危（会写磁盘）。

这个等级信息后面会展示在审批框里，让用户一眼看出当前操作的危险指数。

03、审批请求怎么设计

审批请求是 HITL 系统里用户感知最强的一环——它决定了用户看到什么，进而决定用户能不能做出合理的判断。

ApprovalRequest 的定义如下：

https://paicoding.com/column/17/6

6/18

java 复制代码

```
public record ApprovalRequest(
    String toolName,
    String arguments,
    String dangerLevel,
    String riskDescription,
    String suggestion,
    String callerContext
) {
    public static ApprovalRequest of(String toolName, String arguments, String
suggestion) {
        return new ApprovalRequest(
            toolName,
            arguments,
            ApprovalPolicy.getDangerLevel(toolName),
            ApprovalPolicy.getRiskDescription(toolName),
            suggestion,
            null
        );
    }
}
```

Record 在这里特别合适，因为审批请求是个纯数据载体，不可变，不需要方法逻辑。构建一次就是最终状态。

但光有数据还不够，还需要一个 `toDisplayText()` 方法，把这些字段格式化好看的终端输出：

lua 复制代码



这个 `toDisplayText()` 方法里有个细节值得讲，就是**显示宽度算法**。

终端输出中，中文字符和 emoji 占两列，英文字符占一列。

如果不做处理，直接用字符串长度做 padding，中文和 emoji 就会把右边框挤歪掉，整个审批框看起来就歪歪扭扭的。

所以 `ApprovalRequest` 里实现了一个 `displayWidth()` 方法，专门计算字符串的终端显示列宽：

java 复制代码

```
static int displayWidth(String s) {
    int w = 0;
    int i = 0;
    while (i < s.length()) {
        int cp = s.codePointAt(i);
        i += Character.charCount(cp);
        if (isWideCodePoint(cp)) {
            w += 2; // CJK / 全角 / emoji 占 2 列
        } else {
            w += 1;
        }
    }
    return w;
}
```

CJK 字符（中日韩统一表意文字）、全角符号、常见 emoji，都被识别为宽字符，按 2 列计算。

这样不管参数里有没有中文路径、中文文件名，审批框的右边框始终对齐。

另外参数展示做了 JSON 结构解析。

与其把整个 JSON 字符串糊到框里，不如逐字段展示，`write_file` 的 content 通常很长，超过 120 字符的部分用 `...` 截断并附上总长度。

目录

[01、为什么 Agent 需要人工...](#)

[02、危险操作怎么判断](#)

[03、审批请求怎么设计](#)

[04、审批结果怎么设计](#)

[05、终端交互怎么实现](#)

[06、拦截层：HitlToolRegistry](#)

[07、集成到 Main.java](#)

[08、流式渲染器的冲突处理](#)

[09、HITL 的几个设计权衡](#)

[权衡一：默认关闭](#)

[权衡二：拒绝原因回传给 Agent](#)

[权衡三：APPROVED_ALL 的...](#)

[权衡四：接口隔离测试](#)

[10、单元测试](#)

[11、完整跑一遍](#)

[ending](#)

04、审批结果怎么设计

用户的决策有五种：

java 复制代码

```
public record ApprovalResult(  
    Decision decision,  
    String modifiedArguments,  
    String reason  
) {  
    public enum Decision {  
        APPROVED,           // 批准，用原始参数  
        APPROVED_ALL,       // 本次会话全部放行同类操作  
        REJECTED,           // 拒绝，Agent 会收到拒绝原因  
        MODIFIED,           // 修改参数后执行  
        SKIPPED              // 跳过本步骤  
    }  
}
```

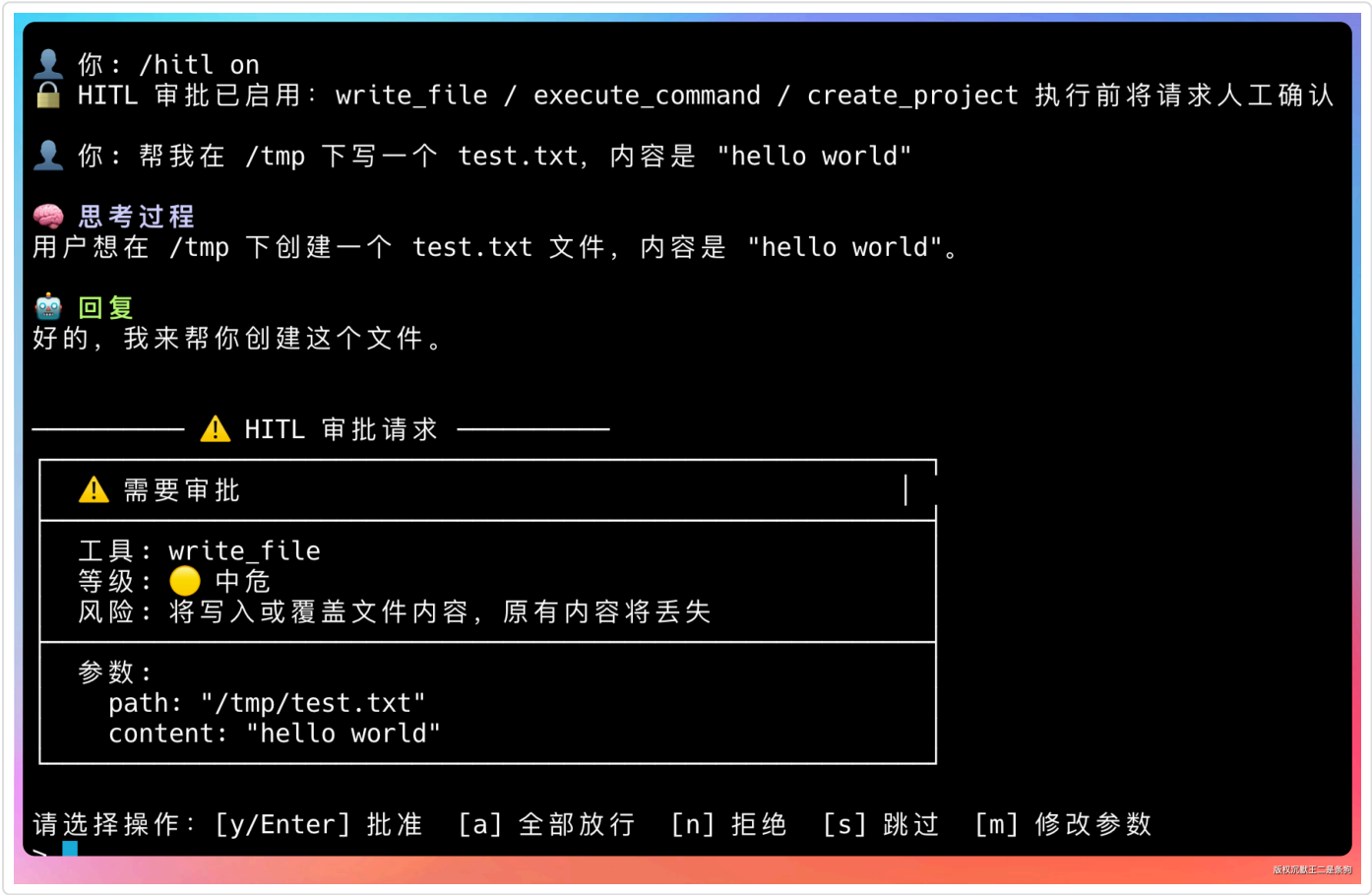
这五种决策覆盖了大多数场景。

APPROVED 是最普通的批准，每次都要确认。

APPROVED_ALL 更方便——如果你正在做一批文件操作，不想每次都确认 **write_file**，按 **a** 就可以把这个工具的后续操作全部放行，直到下次 **/clear**。

MODIFIED 允许用户在批准前修改参数，比如 Agent 想把文件写到 **/tmp/test.json**，你可以改成 **~/project/test.json** 再执行。

REJECTED 和 **SKIPPED** 区别在于语义：拒绝会带上原因让 Agent 重新规划，跳过只是这一步不执行。



effectiveArguments() 方法封装了参数选择逻辑：

java 复制代码

```
public String effectiveArguments(String originalArguments) {  
    if (decision == Decision.MODIFIED && modifiedArguments != null  
        && !modifiedArguments.isBlank()) {  
        return modifiedArguments;  
    }  
    return originalArguments;  
}
```

如果用户选了修改，就用修改后的参数；其他情况一律用原始参数。

05、终端交互怎么实现

HitlHandler 是审批处理的接口，**TerminalHitlHandler** 是终端实现。

目录

- [01、为什么 Agent 需要人工...](#)
- [02、危险操作怎么判断](#)
- [03、审批请求怎么设计](#)
- [04、审批结果怎么设计](#)
- [05、终端交互怎么实现](#)
- [06、拦截层：HitlToolRegistry](#)
- [07、集成到 Main.java](#)
- [08、流式渲染器的冲突处理](#)
- [09、HITL 的几个设计权衡](#)
 - [权衡一：默认关闭](#)
 - [权衡二：拒绝原因回传给 Agent](#)
 - [权衡三：APPROVED_ALL 的...](#)
 - [权衡四：接口隔离测试](#)
- [10、单元测试](#)
- [11、完整跑一遍 ending](#)

接口设计非常简单：

java 复制代码

```
public interface HitlHandler {
    boolean isEnabled();
    void setEnabled(boolean enabled);
    ApprovalResult requestApproval(ApprovalRequest request);
}
```

TerminalHitlHandler 的核心是 requestApproval 方法。整体加了 synchronized。

Multi-Agent 模式下多个 Worker 并发可能会触发危险操作，如果不加同步，两个审批框会同时输出到 stdout，stdin 也会被两个线程抢占，用户会看到的终端输出就乱了。

synchronized 保证审批请求是串行的——一次只展示一个，等用户决策完了，再弹出来下一个。

java 复制代码

```
@Override
public synchronized ApprovalResult requestApproval(ApprovalRequest request) {
    // 如果该工具已在本次会话中被批准"全部放行", 直接通过
    if (approvedAllTools.contains(request.toolName())) {
        out.println(" [HITL] " + request.toolName() + " 已在本次会话中全部放行, 自动通过");
        return ApprovalResult.approveAll();
    }

    out.println();
    out.println("————— ⚠ HITL 审批请求 —————");
    out.println(request.toDisplayText());

    return promptUntilDecision(request);
}
```

注意这里在审批框上方打印了一行分隔符 ————— ⚠ HITL 审批请求 —————。

这是为了解决一个体验问题：Agent 流式输出“回复”内容的时候，Markdown 渲染器有缓冲区。

如果审批框紧贴着上游的 reasoning 文字输出，视觉上很难分辨哪里结束、哪里开始。

分隔符拉开了视觉间距。

主交互循环 promptUntilDecision 有个 fail-safe 设计：连续 5 次无法识别的输入，保守处理为 REJECTED。

java 复制代码

```
private ApprovalResult promptUntilDecision(ApprovalRequest request) {
    for (int attempt = 0; attempt < 5; attempt++) {
        out.println("请选择操作: [y/Enter] 批准 [a] 全部放行 [n] 拒绝 [s] 跳过 [m] 修改参数");
        out.print("> ");
        out.flush();

        String input = in.readLine();
        String normalized = input.trim().toLowerCase();

        if (normalized.isEmpty() || normalized.equals("y")) {
            return ApprovalResult.approve();
        }
        switch (normalized) {
            case "a" -> { /* 全部放行 */ }
            case "n" -> { /* 拒绝并收集原因 */ }
            case "s" -> { return ApprovalResult.skip(); }
            case "m" -> { /* 修改参数流程 */ }
            default -> out.println(" ? 无法识别的选项, 请输入 y/a/n/s/m 之一");
        }
    }
    // 连续多次无效, 保守拒绝
    return ApprovalResult.reject("连续多次无效输入");
}
```

为什么是拒绝，不是默认批准？

因为用户没有明确表示同意就不该执行。如果真遇到什么奇怪的边界情况，宁愿让 Agent 重新规划。

目录

[01、为什么 Agent 需要人工...](#)

[02、危险操作怎么判断](#)

[03、审批请求怎么设计](#)

[04、审批结果怎么设计](#)

[05、终端交互怎么实现](#)

[06、拦截层: HitlToolRegistry](#)

[07、集成到 Main.java](#)

[08、流式渲染器的冲突处理](#)

[09、HITL 的几个设计权衡](#)

[权衡一：默认关闭](#)

[权衡二：拒绝原因回传给 Agent](#)

[权衡三：APPROVED_ALL 的...](#)

[权衡四：接口隔离测试](#)

[10、单元测试](#)

[11、完整跑一遍](#)

[ending](#)

approvedAllTools 用的是 ConcurrentHashMap.newKeySet()，多线程场景下安全。

06、拦截层：HitlToolRegistry

这是整个 HITL 系统最关键的类。

HitlToolRegistry 继承自 ToolRegistry，只覆写了 executeTool 一个方法：

java 复制代码

```
public class HitlToolRegistry extends ToolRegistry {

    private final HitlHandler hitlHandler;

    public HitlToolRegistry(HitlHandler hitlHandler) {
        super();
        this.hitlHandler = hitlHandler;
    }

    @Override
    public String executeTool(String name, String argumentsJson) {
        // HITL 未启用或该工具不需要审批，直接执行
        if (!hitlHandler.isEnabled() || !ApprovalPolicy.requiresApproval(name)) {
            return super.executeTool(name, argumentsJson);
        }

        // 构建审批请求并发起审批
        ApprovalRequest request = ApprovalRequest.of(name, argumentsJson, null);
        ApprovalResult result = hitlHandler.requestApproval(request);

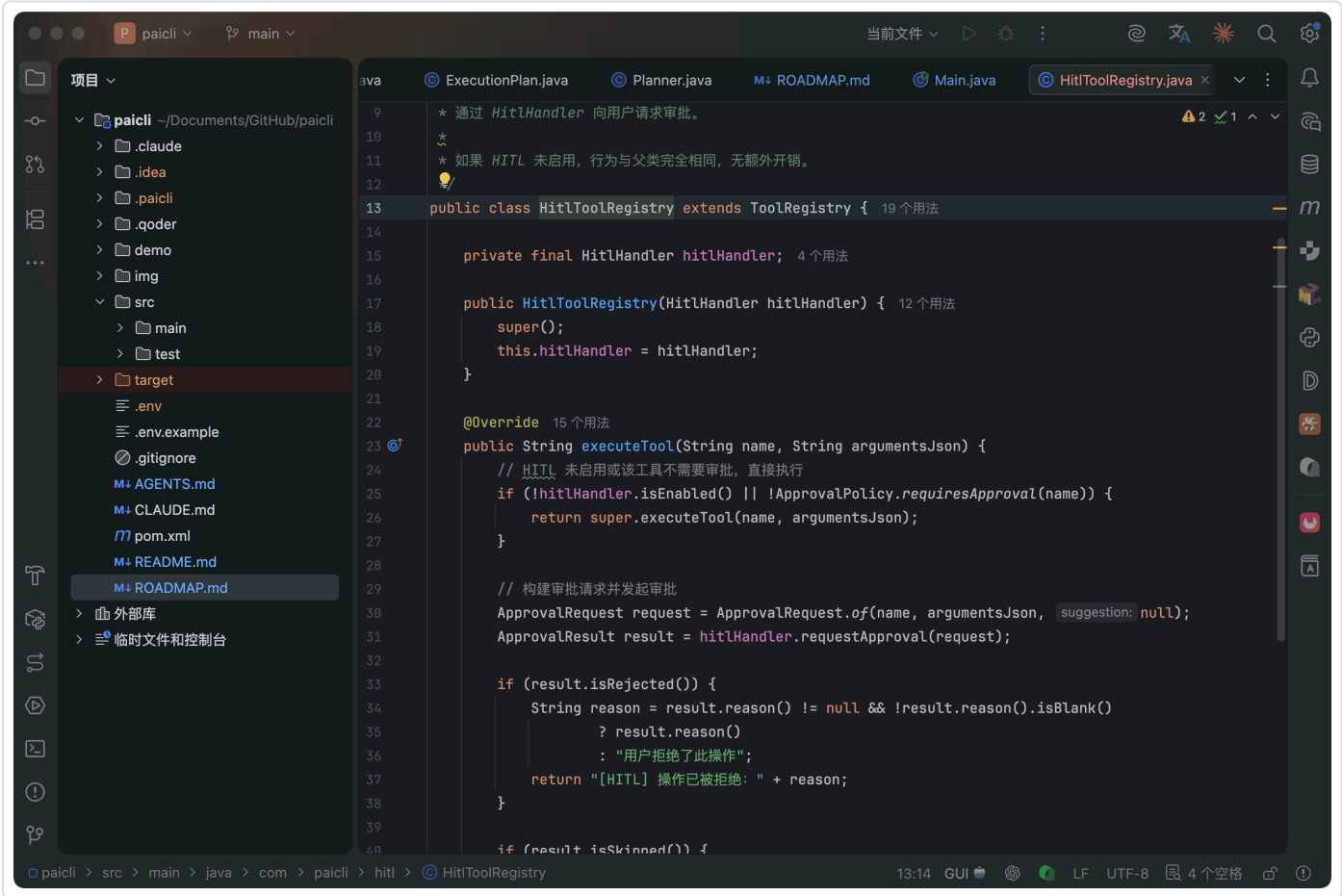
        if (result.isRejected()) {
            String reason = result.reason() != null ? result.reason() : "用户拒绝了此操作";
            return "[HITL] 操作已被拒绝: " + reason;
        }
        if (result.isSkipped()) {
            return "[HITL] 操作已被跳过";
        }

        // 批准（含修改参数）
        String effectiveArgs = result.effectiveArguments(argumentsJson);
        return super.executeTool(name, effectiveArgs);
    }
}
```

整个逻辑就这 20 行。

先检查 HITL 有没有启用，没启用直接 super.executeTool()，和普通的 ToolRegistry 完全一致。

检查这个工具需不需要审批，不需要的话也直接放行。只有“HITL 启用 + 工具危险”这两个条件同时成立，才走审批流程。



目录

[01、为什么 Agent 需要人工...](#)

[02、危险操作怎么判断](#)

[03、审批请求怎么设计](#)

[04、审批结果怎么设计](#)

[05、终端交互怎么实现](#)

[06、拦截层：HitlToolRegistry](#)

[07、集成到 Main.java](#)

[08、流式渲染器的冲突处理](#)

[09、HITL 的几个设计权衡](#)

[权衡一：默认关闭](#)

[权衡二：拒绝原因回传给 Agent](#)

[权衡三：APPROVED_ALL 的...](#)

[权衡四：接口隔离测试](#)

[10、单元测试](#)

[11、完整跑一遍](#)

[ending](#)

如果 HITL 关闭，`hitlHandler.isEnabled()` 返回 `false`，第一个 `if` 就短路了，`super.executeTool()` 直接执行。

运行时的判断开销就是一次 `boolean` 读取 + 一次 `Set` 查找，几乎可以忽略。

这也是继承在这里的价值：不用引入新接口，不用改老代码。

07、集成到 Main.java

`HitlToolRegistry` 在 `Main.java` 里统一初始化：

java 复制代码

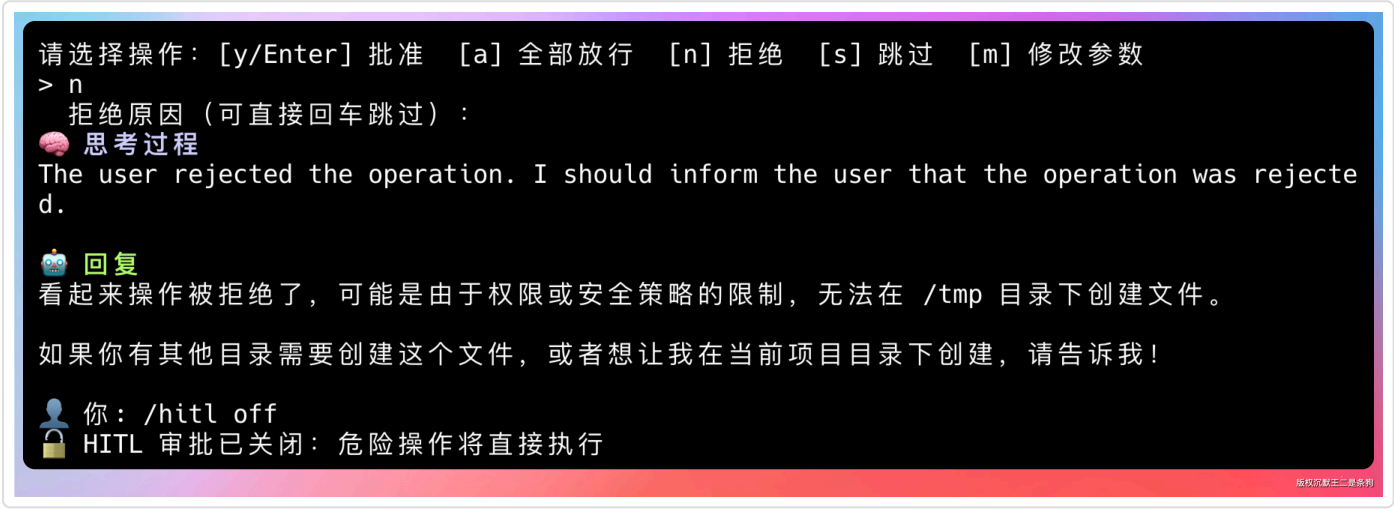
```
TerminalHitlHandler hitlHandler = new TerminalHitlHandler(false); // 默认关闭
HitlToolRegistry toolRegistry = new HitlToolRegistry(hitlHandler);

// 传给 Agent 使用
Agent agent = new Agent(apiKey, toolRegistry);
```

开关通过 `/hitl` 命令控制：

bash 复制代码

```
/hitl on      # 启用
/hitl off    # 关闭
/hitl        # 查看当前状态
```



`/clear` 命令里还加了一个细节：清除本次会话中积累的“全部放行”记录。

java 复制代码

```
case "/clear" -> {
    memory.clear();
    hitlHandler.clearApprovedAll(); // 清除 APPROVED_ALL 记录
    System.out.println("已清除对话历史");
}
```

这个设计很有必要。

用户在一次会话里对 `write_file` 选了“全部放行”，下次 `/clear` 重新开始对话，如果不清除，这个放行记录还在。清除对话历史就应该把审批状态一起清掉，两者在语义上是绑定的。

08、流式渲染器的冲突处理

这个问题我卡了一段时间。

`PaiCLI` 的 `Agent` 在流式输出 `reasoning_content` 的时候，`TerminalMarkdownRenderer` 有缓冲区。渲染器遇到 `\n` 才 `flush` 一行内容。

问题是：`Agent` 在工具调用迭代之间，渲染器里可能还有未 `flush` 的文字缓冲。

如果这时候 HITL 审批框直接弹出来，会从半行文字的中间插入，审批框的标题和内容就错位了。

解决方式是在进入 `tool-call` 迭代前，先调用 `renderer.resetBetweenIterations()`，把缓冲区强制 `flush` 掉：

目录

[01、为什么 Agent 需要人工...](#)

[02、危险操作怎么判断](#)

[03、审批请求怎么设计](#)

[04、审批结果怎么设计](#)

[05、终端交互怎么实现](#)

[06、拦截层：HitlToolRegistry](#)

[07、集成到 Main.java](#)

[08、流式渲染器的冲突处理](#)

[09、HITL 的几个设计权衡](#)

[权衡一：默认关闭](#)

[权衡二：拒绝原因回传给 Agent](#)

[权衡三：APPROVED_ALL 的...](#)

[权衡四：接口隔离测试](#)

[10、单元测试](#)

[11、完整跑一遍](#)

[ending](#)

java 复制代码

```
// Agent.java: 进入工具调用前 flush 渲染缓冲
renderer.resetBetweenIterations();
// 然后再执行工具
String toolResult = toolRegistry.executeTool(toolName, toolArgs);
```

Agent.java、SubAgent.java、PlanExecuteAgent.java 三条路径都做了这个处理。漏掉任何一条，在对应模式下就会出现排版错乱。



这类问题很隐蔽，单测不好覆盖，必须手工跑一遍流式 Agent 对话才能发现。

09、HITL 的几个设计权衡

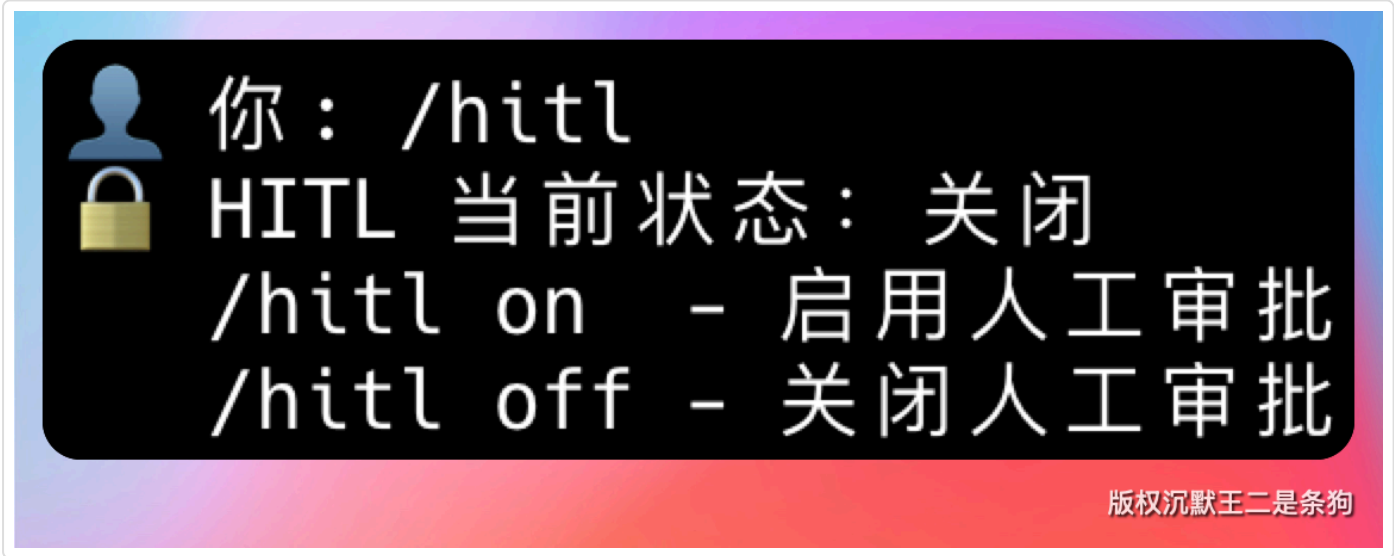
有几个设计选择值得展开说说。

权衡一：默认关闭

HITL 默认是关闭的，要手动 `/hitl on` 开启。

可能大家会觉得，危险功能保护不是应该默认开启吗？我考虑过，但最后决定还是默认关闭。

原因是：开发和调试阶段，频繁的审批确认会打断节奏。



你知道 Agent 要干什么，你也信任这个操作，但每次都要手动按 `y`，会很烦。HITL 适合的场景是——你不太确定 Agent 会做什么，或者操作结果不可逆，才有价值。

默认关闭让用户主动选择，而不是强迫所有人都走审批流程。

权衡二：拒绝原因回传给 Agent

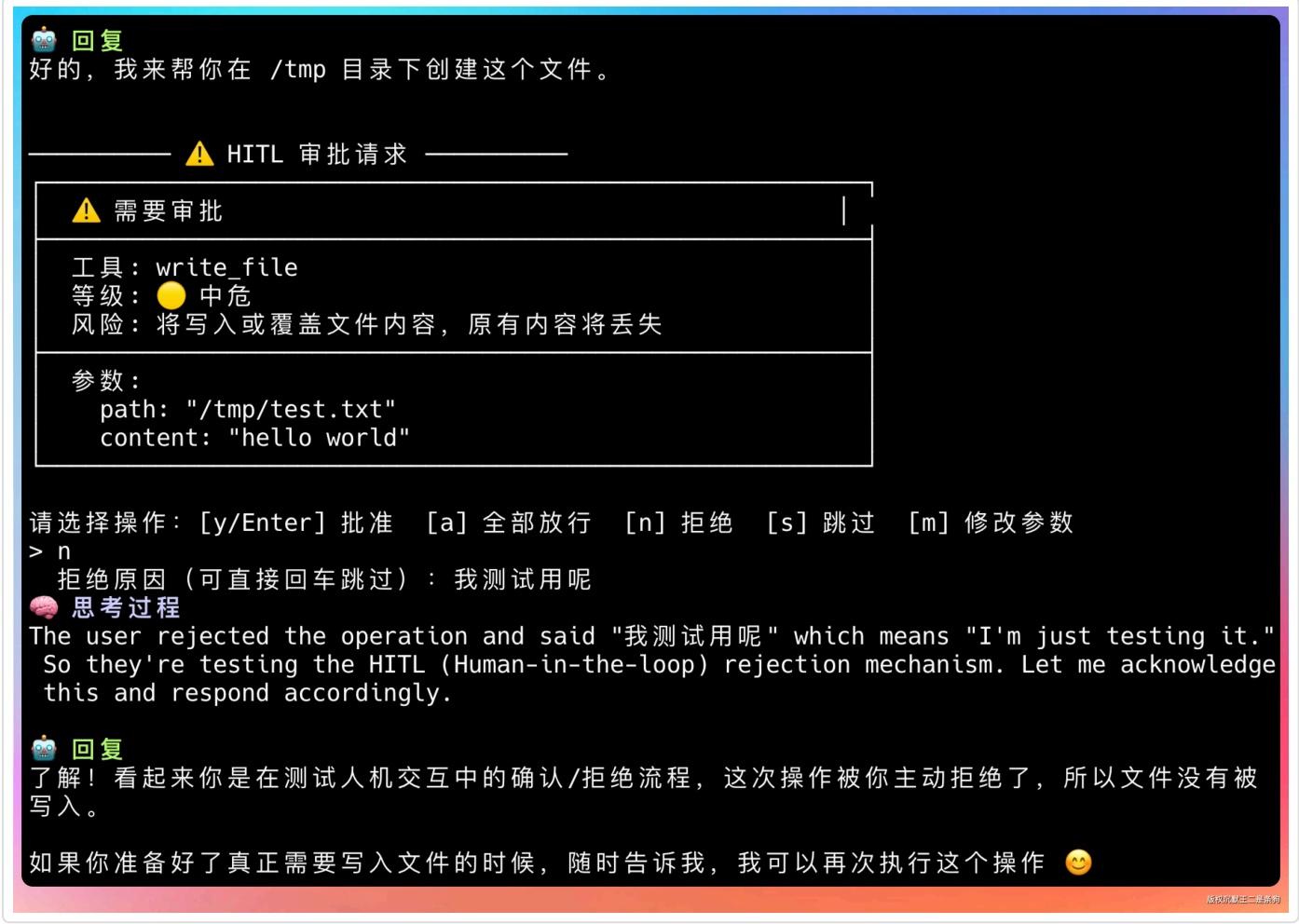
用户选 `n` 拒绝的时候，拒绝原因会作为工具调用结果返回给 Agent：

bash 复制代码

```
[HITL] 操作已被拒绝：路径有误，应该写到 ~/project 而不是 /tmp
```

目录

- [01、为什么 Agent 需要人工...](#)
- [02、危险操作怎么判断](#)
- [03、审批请求怎么设计](#)
- [04、审批结果怎么设计](#)
- [05、终端交互怎么实现](#)
- [06、拦截层：HitlToolRegistry](#)
- [07、集成到 Main.java](#)
- [08、流式渲染器的冲突处理](#)
- [09、HITL 的几个设计权衡](#)
 - [权衡一：默认关闭](#)
 - [权衡二：拒绝原因回传给 Agent](#)
 - [权衡三：APPROVED_ALL 的...](#)
 - [权衡四：接口隔离测试](#)
- [10、单元测试](#)
- [11、完整跑一遍 ending](#)



Agent 看到这个结果，可以调整思路重新规划。

这是比“默默不执行”更好的设计——Agent 知道为什么被拒绝，而不是一头雾水地原地等待或者无限重试同一个操作。

权衡三：APPROVED_ALL 的作用域是工具级别，不是全局

“全部放行”是按工具名作为 key 存储的，放行了 `write_file` 不代表放行 `execute_command`。

这个粒度是有意为之的。文件写入和命令执行的风险差距很大，用户可能愿意放行一批文件操作，但对执行命令还是想逐一审核。

权衡四：接口隔离测试

`HitlHandler` 是接口，`TerminalHitlHandler` 是实现。测试里用的是 Mockito 的 mock，不需要真实的终端 `stdin/stdout`。

这个接口分离不是为了“设计模式好看”，是为了让测试能跑通。终端交互是一个非常难测的东西，一旦代码里硬编码了 `System.in`，测试就必须模拟键盘输入，极其麻烦。

接口抽象之后，`TerminalHitlHandler` 的包级别构造器允许注入自定义的 `BufferedReader` 和 `PrintStream`，测试可以直接喂字符串：

```
java 复制代码
// 测试代码：注入 "y\n" 模拟用户输入 y
BufferedReader mockIn = new BufferedReader(new StringReader("y\n"));
ByteArrayOutputStream out = new ByteArrayOutputStream();
TerminalHitlHandler handler = new TerminalHitlHandler(true, mockIn, new PrintStream(out));
```

10、单元测试

三个测试类：`ApprovalPolicyTest`、`ApprovalResultTest`、`HitlToolRegistryTest`。

`ApprovalPolicyTest` 验证策略正确性——哪些工具需要审批，哪些不需要，危险等级是否正确：

目录

[01、为什么 Agent 需要人工...](#)

[02、危险操作怎么判断](#)

[03、审批请求怎么设计](#)

[04、审批结果怎么设计](#)

[05、终端交互怎么实现](#)

[06、拦截层：HitlToolRegistry](#)

[07、集成到 Main.java](#)

[08、流式渲染器的冲突处理](#)

[09、HITL 的几个设计权衡](#)

[权衡一：默认关闭](#)

[权衡二：拒绝原因回传给 Agent](#)

[权衡三：APPROVED_ALL 的...](#)

[权衡四：接口隔离测试](#)

[10、单元测试](#)

[11、完整跑一遍](#)

[ending](#)

java 复制代码

```
@Test
void testRequiresApproval() {
    assertTrue(ApprovalPolicy.requiresApproval("write_file"));
    assertTrue(ApprovalPolicy.requiresApproval("execute_command"));
    assertTrue(ApprovalPolicy.requiresApproval("create_project"));
    assertFalse(ApprovalPolicy.requiresApproval("read_file"));
    assertFalse(ApprovalPolicy.requiresApproval("list_dir"));
    assertFalse(ApprovalPolicy.requiresApproval("search_code"));
}
```

HitlToolRegistryTest 最关键——用 Mock 的方式验证拦截逻辑：

java 复制代码

```
@Test
void testDangerousToolIsIntercepted() {
    HitlHandler mockHandler = mock(HitlHandler.class);
    when(mockHandler.isEnabled()).thenReturn(true);
    when(mockHandler.requestApproval(any())).thenReturn(ApprovalResult.approve());

    HitlToolRegistry registry = new HitlToolRegistry(mockHandler);
    // 执行危险工具
    registry.executeTool("write_file", "{\"path\":\"test.txt\",\"content\":\"hello\"}");

    // 验证审批被调用了一次
    verify(mockHandler, times(1)).requestApproval(any());
}

@Test
void testSafeToolBypassesHitl() {
    HitlHandler mockHandler = mock(HitlHandler.class);
    when(mockHandler.isEnabled()).thenReturn(true);

    HitlToolRegistry registry = new HitlToolRegistry(mockHandler);
    registry.executeTool("read_file", "{\"path\":\"test.txt\"}");

    // 安全工具不触发审批
    verify(mockHandler, never()).requestApproval(any());
}
```

目录

01、为什么 Agent 需要人工...

02、危险操作怎么判断

03、审批请求怎么设计

04、审批结果怎么设计

05、终端交互怎么实现

06、拦截层：HitlToolRegistry

07、集成到 Main.java

08、流式渲染器的冲突处理

09、HITL 的几个设计权衡

 权衡一：默认关闭

 权衡二：拒绝原因回传给 Agent

 权衡三：APPROVED_ALL 的...

 权衡四：接口隔离测试

10、单元测试

11、完整跑一遍

ending



全部 152 个测试通过，HITL 相关的测试零故障。

11、完整跑一遍

启动 PaiCLI 之后，先开启 HITL：

shell 复制代码

```
> /hitl on
✅ HITL 已启用，危险操作将在执行前请求确认
```

然后让 Agent 写一个文件：

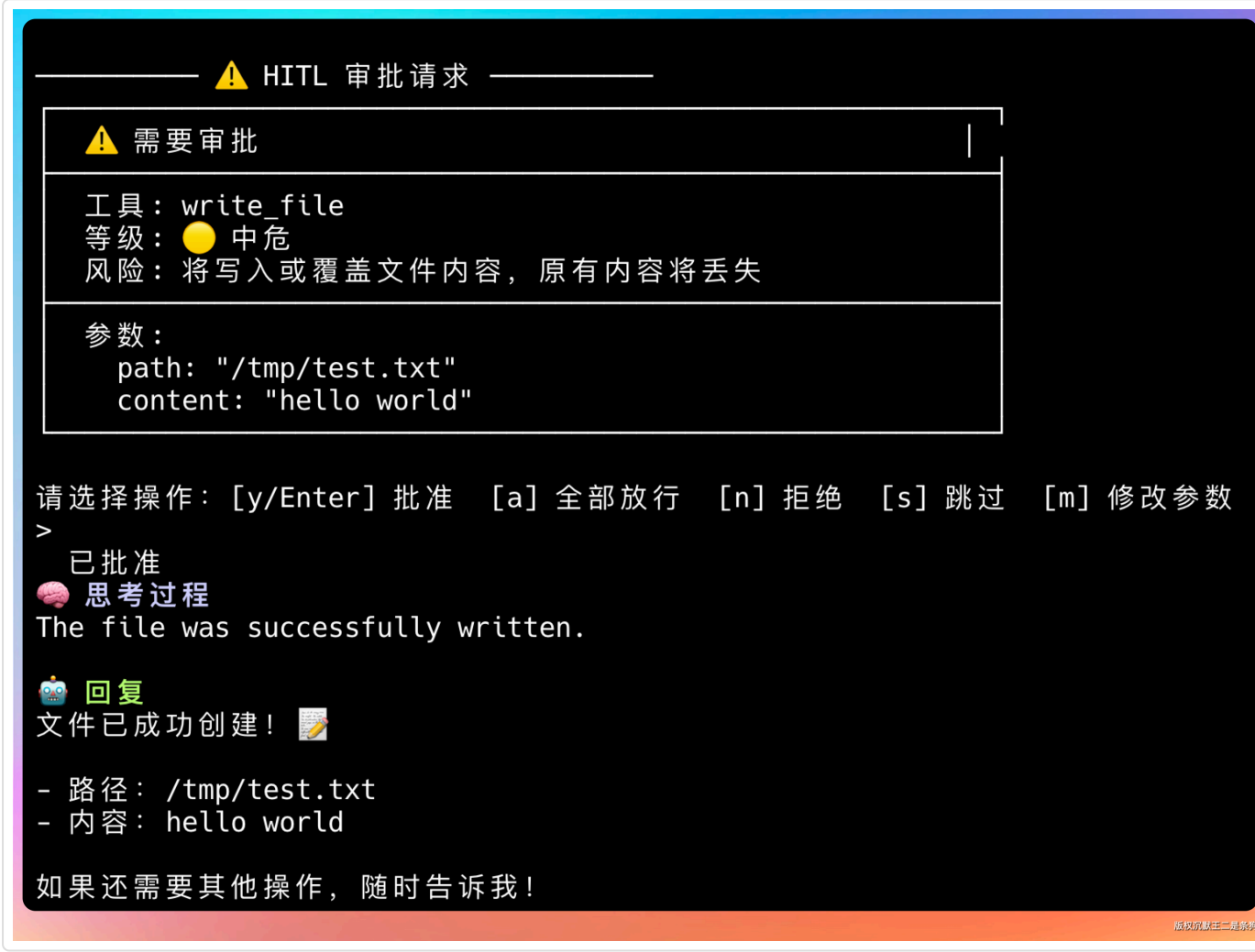
shell 复制代码

> 帮我在 /tmp 下写一个 test.txt，内容是 "hello world"

Agent 思考完，触发 `write_file` 工具的时候，屏幕上会出现：



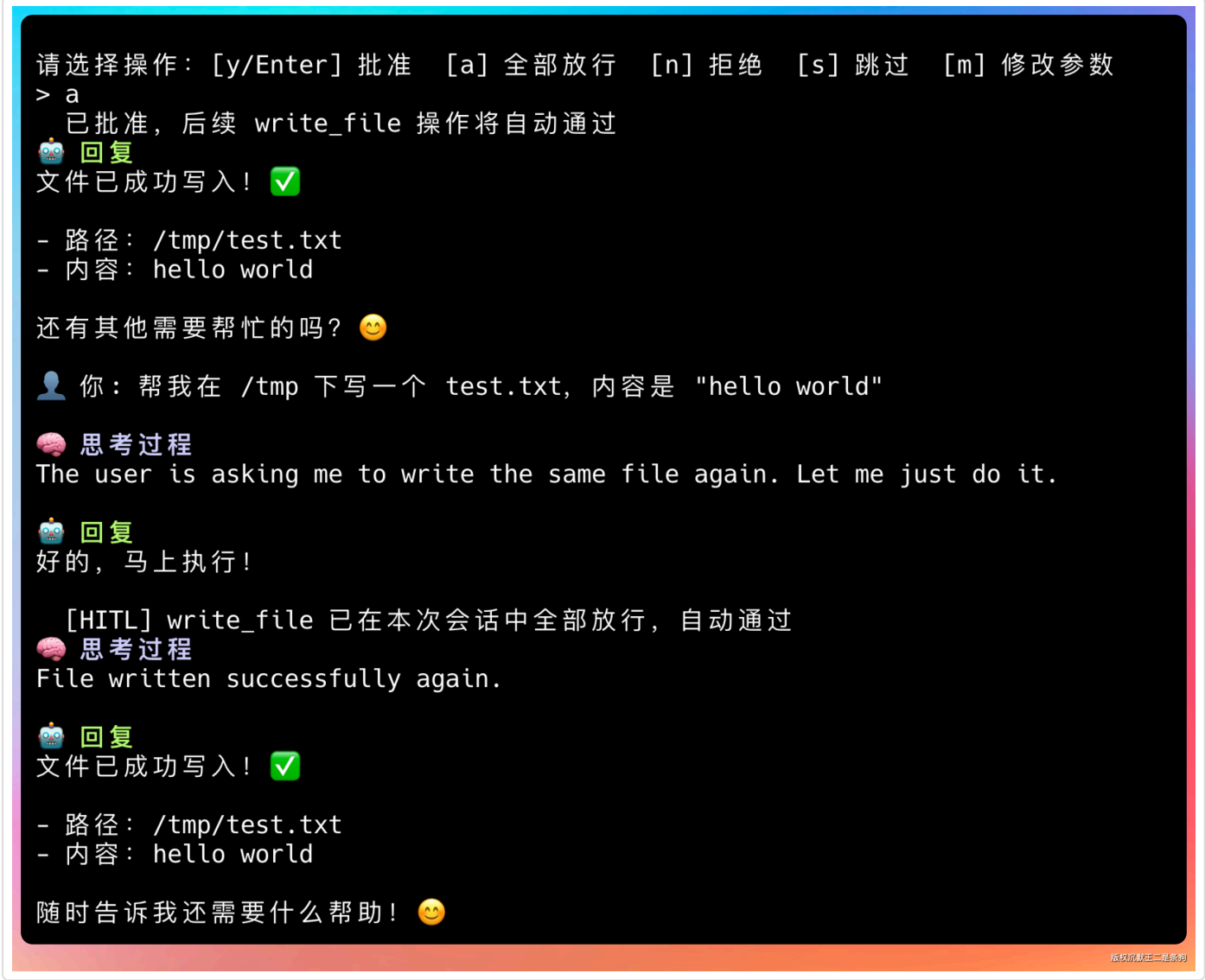
输入 `y` 批准，Agent 继续执行。输入 `n` 拒绝，Agent 收到拒绝反馈并重新规划。



这就是 HITL 的完整体验。

目录

- [01、为什么 Agent 需要人工...](#)
- [02、危险操作怎么判断](#)
- [03、审批请求怎么设计](#)
- [04、审批结果怎么设计](#)
- [05、终端交互怎么实现](#)
- [06、拦截层：HitlToolRegistry](#)
- [07、集成到 Main.java](#)
- [08、流式渲染器的冲突处理](#)
- [09、HITL 的几个设计权衡](#)
 - [权衡一：默认关闭](#)
 - [权衡二：拒绝原因回传给 Agent](#)
 - [权衡三：APPROVED_ALL 的...](#)
 - [权衡四：接口隔离测试](#)
- [10、单元测试](#)
- [11、完整跑一遍 ending](#)



如果不想每次都手动确认，对一批已知安全的 `write_file` 操作可以输入 `a`，后续的 `write_file` 调用会自动放行，直到 `/clear`。

ending

六期做下来，PaiCLI 从一个只会“思考-行动”的 ReAct 小 Agent，逐渐进化到有计划、有记忆、能检索、能协作、会控制风险的 Agent 系统。

很多人第一反应是“让 LLM 来判断危不危险，不是更智能吗”。但智能不等于可靠。工具执行是确定性的，判断“是否要阻断”这件事，也应该是确定的。

简单的东西往往比聪明的东西更靠谱。一行 `Set.contains()` 比一次 LLM 调用更可信。

目录

- [01、为什么 Agent 需要人工...](#)
- [02、危险操作怎么判断](#)
- [03、审批请求怎么设计](#)
- [04、审批结果怎么设计](#)
- [05、终端交互怎么实现](#)
- [06、拦截层：HitlToolRegistry](#)
- [07、集成到 Main.java](#)
- [08、流式渲染器的冲突处理](#)
- [09、HITL 的几个设计权衡](#)
 - [权衡一：默认关闭](#)
 - [权衡二：拒绝原因回传给 Agent](#)
 - [权衡三：APPROVED_ALL 的...](#)
 - [权衡四：接口隔离测试](#)
- [10、单元测试](#)
- [11、完整跑一遍 ending](#)



目录

[01、为什么 Agent 需要人工...](#)

[02、危险操作怎么判断](#)

[03、审批请求怎么设计](#)

[04、审批结果怎么设计](#)

[05、终端交互怎么实现](#)

[06、拦截层：HitlToolRegistry](#)

[07、集成到 Main.java](#)

[08、流式渲染器的冲突处理](#)

[09、HITL 的几个设计权衡](#)

[权衡一：默认关闭](#)

[权衡二：拒绝原因回传给 Agent](#)

[权衡三：APPROVED_ALL 的...](#)

[权衡四：接口隔离测试](#)

[10、单元测试](#)

[11、完整跑一遍](#)

[ending](#)

简历包装

项目名称：PaiCLI — Java Agent CLI

项目简介：基于 ReAct 范式从零实现的 Java Agent 命令行工具，集成 Plan-and-Execute、Memory、RAG、Multi-Agent 和 HITL 人工审批，完整覆盖 AI Agent 核心技术栈。

技术栈：Java 17、Maven、GLM-5.1 大模型、Jackson、JLine、SQLite（向量存储）、JUnit 5、Mockito

核心职责：

- 基于静态规则实现 HITL 危险操作拦截策略（ApprovalPolicy），将 write_file、execute_command、create_project 标记为需要人工确认的工具，避免 Agent 无监督覆写文件或执行 shell 命令
- 设计并实现继承式拦截层，通过覆写 executeTool() 注入审批逻辑，HITL 关闭时零开销，共享于 ReAct、Plan-and-Execute 和 Multi-Agent 三条执行路径
- 使用 Record 定义数据类型，实现显示宽度算法（CJK/全角/emoji 按 2 列计算）确保中文内容在 ASCII 审批框中渲染对齐
- 通过 synchronized + ConcurrentHashMap 保证多 Agent 并发场景下审批请求串行展示，支持“全部放行”会话级缓存降低重复确认频率
- 使用 Mockito 验证拦截逻辑、策略覆盖率和 fail-safe 行为，随项目总测试用例数增至 152 个



真诚点赞 诚不我欺



讨论应以学习和精进为目的。请勿发布不友善或者负能量的内容，与人为善，比聪明更重要！



0/512

评论

目录

[01、为什么 Agent 需要人工...](#)

[02、危险操作怎么判断](#)

[03、审批请求怎么设计](#)

[04、审批结果怎么设计](#)

[05、终端交互怎么实现](#)

[06、拦截层：HitlToolRegistry](#)

[07、集成到 Main.java](#)

[08、流式渲染器的冲突处理](#)

[09、HITL 的几个设计权衡](#)

[\[权衡一：默认关闭\]\(#\)](#)

[\[权衡二：拒绝原因回传给 Agent\]\(#\)](#)

[\[权衡三：APPROVED_ALL 的...\]\(#\)](#)

[\[权衡四：接口隔离测试\]\(#\)](#)

[10、单元测试](#)

[11、完整跑一遍](#)

[ending](#)